

HO CHI MINH CITY UNIVERSITY OF SCIENCE
FACULTY OF ELECTRONICS AND
TELECOMMUNICATIONS

Design of an I3C Communication Controller

Graduation Thesis

Student: Vo Minh Huy
Student ID: 22207042
Class: 22DVT_CLC1
Supervisor: Nguyen Duy Manh Thi
Credits: 10

Ho Chi Minh City, May 2026

Abstract

This thesis presents the design and verification of a simplified MIPI I3C Basic v1.1.1 master controller implemented in SystemVerilog. The design operates in Single Data Rate (SDR) mode at up to 12.5 MHz, supports Dynamic Address Assignment (DAA) via the ENTDAAs protocol, performs private read/write transfers, and maintains I2C Fast Mode (400 kHz) backward compatibility. The controller is derived from the open-source CHIPS Alliance *i3c-core* reference implementation through a study-and-improve methodology, achieving approximately 92% code reduction — from roughly 25 000 to 4 100 lines of RTL — while retaining full functional coverage of the specified feature set.

A UVM-based functional verification environment is developed alongside the RTL. A systematic bug analysis identifies 22 issues across 18 source files, including 3 critical defects, each with documented root-cause analysis and remediation. The resulting design demonstrates the complete frontend IC design pipeline from specification study through RTL implementation and functional verification.

Keywords: I3C, MIPI, SystemVerilog, RTL design, SDR mode, ENTDAAs, UVM, functional verification, master controller, HCI queues.

Contents

Abstract	i
1 Introduction	1
1.1 Motivation	1
1.2 Objectives	1
1.3 Scope	2
1.4 Methodology	2
1.5 Thesis Organization	3
2 Background: MIPI I3C Protocol	4
2.1 Overview	4
2.2 Bus Topology	4

2.3	I3C versus I2C	4
2.4	SDR Frame Format	5
2.5	Bus Conditions	6
2.6	Timing Parameters	6
2.7	Common Command Codes	6
2.8	Dynamic Address Assignment (ENTDAA)	7
2.9	Private Transfers	7
2.9.1	Private Write	7
2.9.2	Private Read	8
3	Reference Design Analysis	9
3.1	CHIPS Alliance i3c-core	9
3.2	Module-by-Module Strategy	9
3.3	Critical Problems in the Reference Design	10
3.4	Code Reduction Summary	11
4	System Architecture	12
4.1	Top-Level Block Diagram	12
4.2	Register Interface	13
4.3	Register Map	13
4.4	HCI Queue Architecture	14
4.5	Command and Response Descriptors	14
4.5.1	Command Descriptor (64-bit)	14
4.5.2	Response Descriptor (32-bit)	15
4.6	Signal Conventions	15
5	RTL Implementation	16
5.1	Overview	16
5.2	Shared Type Packages	16
5.2.1	i3c_pkg.sv	16
5.2.2	controller_pkg.sv	17
5.3	Physical Layer	17
5.4	SCL Generator	18
5.5	Bus TX and RX Paths	18
5.5.1	TX Path	18
5.5.2	RX Path	18
5.6	HCI Queues	19
5.7	CSR Register File	19
5.8	ENTDAA Engine	19
5.8.1	entdaa_fsm.sv	19

5.8.2	entdaa_controller.sv	20
5.9	Command FSM (flow_active.sv)	20
5.10	Controller Wrapper (controller_active.sv)	22
6	Functional Verification	23
6.1	Verification Strategy	23
6.2	Environment Architecture	23
6.3	Register Agent	23
6.4	I3C Bus Agent	24
6.5	Scoreboard	24
6.6	Phase 1 Test Plan	25
6.7	Build Infrastructure	25
6.8	Phase 2 Roadmap	26
7	Bug Analysis and Fixes	27
7.1	Overview	27
7.2	Critical Issues	27
7.3	High-Severity Issues	28
7.4	Medium-Severity Issues	30
7.5	Low-Severity Issues	31
7.6	Remediation Priority	31
8	Results and Discussion	32
8.1	Design Metrics	32
8.2	Design Contributions	32
8.2.1	Complete Command FSM	32
8.2.2	Register File Redesign	33
8.2.3	OD/PP Driver Switching	33
8.2.4	Master-Side ENTDAAs FSM	33
8.2.5	Infrastructure Simplification	33
8.2.6	Systematic Bug Documentation	33
8.3	Architectural Trade-offs	34
8.4	Limitations	34
9	Conclusion	35
9.1	Summary	35
9.2	Future Work	35

List of Figures

4.1	Top-level data flow diagram	12
5.1	SCL generator 13-state FSM	18
6.1	UVM environment architecture	23

List of Tables

2.1	Comparison of I2C Fast-Mode Plus and I3C SDR	5
2.2	I3C bus conditions	6
2.3	I3C SDR timing defaults at 333 MHz system clock	6
2.4	Implemented Common Command Codes	7
3.1	Thesis strategy per reference module	10
3.2	Code reduction: reference vs. thesis	11
4.1	Register bus interface signals	13
4.2	CSR register map	13
4.3	HCI FIFO configuration	14
4.4	CMD FIFO descriptor field encoding	14
4.5	RESP FIFO descriptor field encoding	15
5.1	RTL source file inventory	16
5.2	Master-side ENTDAAs FSM states	20
5.3	flow_active FSM state responsibilities	21
6.1	Register agent components (Phase 1, implemented)	24
6.2	Phase 1 test scenarios	25
6.3	Phase 2 verification roadmap	26
7.1	Bug severity summary	27

8.1	Design metrics summary	32
8.2	Key architectural decisions and trade-offs	34

Chapter 1. Introduction

1.1 Motivation

Modern System-on-Chip (SoC) and embedded system designs increasingly demand high-speed, low-power serial communication capable of connecting many peripheral devices on a shared two-wire bus. The Inter-Integrated Circuit (I2C) bus, while ubiquitous, has well-documented limitations: a maximum speed of 1 MHz (Fast-Mode Plus), static address assignment that creates address conflicts, and open-drain signaling that wastes power through continuous pull-up resistor current.

The MIPI Alliance introduced the I3C (Improved Inter-Integrated Circuit) bus standard to address these deficiencies. I3C retains the two-wire topology and full I2C backward compatibility while delivering a 12.5-fold increase in maximum clock frequency, push-pull signaling for lower power, and a dynamic address assignment protocol that eliminates static address conflicts.

The design of an I3C master controller provides a comprehensive exercise in the frontend digital IC design pipeline: specification study, architectural decomposition, RTL implementation in SystemVerilog, and functional verification using UVM. This pipeline mirrors industry practice and is directly relevant to careers in integrated circuit design and verification.

1.2 Objectives

This thesis aims to:

1. Develop a thorough understanding of the MIPI I3C Basic v1.1.1 specification, with focus on SDR mode, the ENTDAAs dynamic address assignment protocol, private read/write transactions, and I2C backward compatibility.
2. Design an I3C master controller at the RTL level using SystemVerilog, derived from and improving upon the open-source CHIPS Alliance i3c-core reference implementation.
3. Build a UVM-based functional verification environment to validate all specified transfer modes.
4. Document design decisions, architectural improvements over the reference, and identified bugs with their remediation.

1.3 Scope

In Scope

- SDR mode operation up to 12.5 MHz
- Dynamic Address Assignment via ENTDAAs
- Private write and read transfers (immediate and regular)
- I2C Fast Mode backward compatibility (400 kHz, hardcoded timing)
- Essential Common Command Codes: ENTDAAs (0x07), ENECs (0x00/0x80), DISECs (0x01/0x81)
- Master (Active Controller) role only
- RTL-level design; full physical implementation is out of scope

Explicitly Excluded

- In-Band Interrupts (IBI) and Hot-Join
- HDR modes (HDR-DDR, HDR-TSL, HDR-TSP)
- Multi-master / secondary controller operation
- Target (slave) mode
- Bus recovery protocol
- Full MIPI HCI compliance

1.4 Methodology

This thesis adopts a **study-and-improve** methodology. The starting point is the CHIPS Alliance i3c-core reference implementation — a production-grade I3C controller designed for the Caliptra root-of-trust subsystem. The process follows four steps:

1. **Study** — analyze the reference design’s architecture, module decomposition, FSM structures, and protocol handling.
2. **Extract** — identify the subset of modules and logic relevant to the basic master controller scope.

3. **Simplify** — eliminate complexity introduced by Caliptra-specific requirements (AXI adapters, target mode, IBI, auto-generated CSR).
4. **Improve** — complete unimplemented FSM states, fix known architectural issues, implement proper OD/PP switching, and write a clean hand-crafted register file.

This approach grounds the design in a proven architecture while producing original contributions in completion, simplification, and focused verification.

1.5 Thesis Organization

Chapter 2 covers the MIPI I3C protocol: bus topology, SDR frame format, bus conditions, timing parameters, CCC commands, and the ENTDA sequence.

Chapter 3 analyzes the CHIPS Alliance i3c-core reference and identifies reuse, simplification, and improvement opportunities.

Chapter 4 describes the proposed system architecture and module decomposition.

Chapter 5 documents the RTL implementation of each module.

Chapter 6 describes the UVM verification environment and test plan.

Chapter 7 presents the systematic bug analysis: 22 issues across 18 source files.

Chapter 8 discusses results and evaluates the design against objectives.

Chapter 9 concludes with a summary and future work directions.

Chapter 2. Background: MIPI I3C Protocol

2.1 Overview

I3C (Improved Inter-Integrated Circuit) is a serial communication bus standard defined by the MIPI Alliance. It is designed to unify sensor and actuator communication in modern SoC designs, replacing I2C and SPI with a single, faster, lower-power interface while preserving backward compatibility with existing I2C devices. The target specification for this thesis is **MIPI I3C Basic v1.1.1 with Errata 01 (2022)**.

2.2 Bus Topology

I3C uses a shared two-wire bus: **SCL** (Serial Clock), driven exclusively by the master in SDR mode, and **SDA** (Serial Data), which is bidirectional. Both lines have an internal pull-up to VDD; unlike I2C, no external pull-up resistor is required for I3C-native devices.

The bus supports two signaling modes:

- **Open-drain**: used after START (address phase) and for all I2C legacy communication. Allows wired-AND arbitration between multiple drivers.
- **Push-pull**: used during I3C SDR data phases for maximum throughput (12.5 MHz). Requires the controller to switch the PHY driver mode.

2.3 I3C versus I2C

Table 2.1 summarizes the key differences between I2C Fast-Mode Plus and I3C SDR mode.

Table 2.1: Comparison of I2C Fast-Mode Plus and I3C SDR

Feature	I2C (Fm+)	I3C (SDR)
Max clock frequency	1 MHz	12.5 MHz
Max data rate	1 Mbit/s	12.5 Mbit/s
Signaling	Open-Drain only	OD + Push-Pull
Address space	7-bit, static	7-bit, dynamic
Address assignment	Fixed / SW	Dynamic (ENTDAA)
Clock stretching	Yes (target)	Not permitted (target)
Pull-up resistor	External	Internal
Backward compat.	N/A	Full I2C Fm+
Error detection	ACK/NACK	Parity (T-bit)

2.4 SDR Frame Format

An I3C SDR transfer frame has the structure:

```
[START] [Addr(7-bit) + RnW(1-bit) + T-bit] [Data(8-bit) + T-bit] ... [STOP]
```

Listing 2.1: I3C SDR frame format

Each byte is 9 bits: 8 data bits followed by a **T-bit** (Transition bit). The T-bit semantics differ by phase:

- **Address byte T-bit:** ACK/NACK from the target (0 = ACK, 1 = NACK).
- **Write data byte T-bit:** Odd parity over the 8 data bits, computed by the master and verified by the target.
- **Read data byte T-bit:** End-of-data indicator from the target (0 = last byte, 1 = more data to follow).

All transfers are MSB-first. The signaling mode transitions from open-drain (address phase) to push-pull (data phase) after the repeated START.

2.5 Bus Conditions

Table 2.2: I3C bus conditions

Condition	Definition
START (S)	SDA falls while SCL is HIGH
REPEATED START (Sr)	START issued without a preceding STOP
STOP (P)	SDA rises while SCL is HIGH
Bus Idle	Both SCL and SDA HIGH; no active transfer

2.6 Timing Parameters

The controller requires a system clock of at least **333 MHz** to satisfy the most demanding I3C timing constraint: $t_{SCO,max} = 12\text{ ns}$ (clock-to-data output). At 333 MHz, four pipeline stages consume $4 \times 3\text{ ns} = 12\text{ ns}$.

Table 2.3: I3C SDR timing defaults at 333 MHz system clock

Parameter	Symbol	Minimum	Cycles (333 MHz)
SCL LOW period	t_{LOW}	24 ns	8
SCL HIGH period	t_{HIGH}	24 ns	8
Rise time	t_R	—	4
Fall time	t_F	—	4
START setup time	$t_{SU,STA}$	—	8
START hold time	$t_{HD,STA}$	—	8
STOP setup time	$t_{SU,STO}$	12 ns	4

2.7 Common Command Codes

CCCs are issued using the reserved broadcast address `7'h7E`. This thesis implements only three essential CCCs, as shown in Table 2.4.

Table 2.4: Implemented Common Command Codes

Code	Mnemonic	Type	Description
0x00	ENEC	Broadcast	Enable events on all targets
0x01	DISEC	Broadcast	Disable events on all targets
0x07	ENTDAA	Broadcast	Enter Dynamic Address Assignment
0x80	ENEC	Direct	Enable events (specific target)
0x81	DISEC	Direct	Disable events (specific target)

2.8 Dynamic Address Assignment (ENTDAA)

DAA is the mechanism by which the master assigns 7-bit dynamic addresses to unaddressed I3C targets at runtime. Each target has a unique 64-bit identity:

- **48-bit Provisioned ID (PID)**: manufacturer ID (15 bit), type flag (1 bit), part ID (16 bit), instance ID (4 bit), vendor-defined (12 bit).
- **8-bit BCR** (Bus Characteristics Register): capability flags.
- **8-bit DCR** (Device Characteristics Register): device class.

The ENTDAAs sequence proceeds as follows:

1. Master sends [S] 0x7E + W (broadcast header); all unaddressed targets ACK.
2. Master sends ENTDAAs CCC code (0x07); targets ACK.
3. **For each unaddressed target:**
 - (a) Master issues [Sr] 0x7E + R (repeated START, read direction).
 - (b) All unaddressed targets simultaneously drive their 64-bit identity bit-by-bit (PID then BCR then DCR), *with no ACK between fields*.
 - (c) **Arbitration**: a target that drives HIGH (1) but reads LOW (0) on SDA has lost arbitration and withdraws, leaving exactly one winner.
 - (d) Master transmits {7-bit addr, odd_parity}; winning target ACKs.
4. Master issues STOP when all targets are addressed.

2.9 Private Transfers

2.9.1 Private Write

```
[S] [0x7E + W] [ACK] [Sr] [DynAddr + W] [ACK] [Data0+T] ... [
    DataN+T] [P]
```

Listing 2.2: I3C private write frame

The broadcast header is optional for pure I3C buses. The data phase uses push-pull signaling; the master computes the T-bit as odd parity over each byte.

2.9.2 Private Read

```
[S] [DynAddr + R] [ACK] [Data0+T=1] [Data1+T=1] ... [DataN+T
    =0] [P]
```

Listing 2.3: I3C private read frame

The target signals end-of-data by asserting T-bit = 0 on the last byte. Signaling transitions from open-drain (address phase) to push-pull (data phase) after Sr.

Chapter 3. Reference Design Analysis

3.1 CHIPS Alliance i3c-core

The starting point for this thesis is the open-source **CHIPS Alliance i3c-core** — a production-grade I3C controller and target IP designed for integration into the Caliptra root-of-trust subsystem [4]. The reference provides a fully verified, dual-role (controller + target) implementation.

Repository scale: approximately 25 000 lines of SystemVerilog across 80+ source files, with an additional 14 000 lines of auto-generated CSR code.

The top-level hierarchy of the reference design is:

```
i3c.sv
+-- i3c_wrapper.sv          (AHB-Lite / AXI4 frontend)
    +-- I3CCSR.sv           (auto-generated register file,
        7710 lines)
    +-- controller.sv
        |   +-- controller_active.sv
        |   |   +-- flow_active.sv          (command flow FSM)
        |   |   +-- entdaa_controller.sv
        |   |   +-- entdaa_fsm.sv
        |   |   +-- bus_tx_flow.sv
        |   |   +-- bus_rx_flow.sv
        |   |   +-- ibi.sv
        |   +-- bus_monitor.sv
    +-- i3c_phy.sv
    +-- recovery_handler.sv
```

Listing 3.1: CHIPS Alliance i3c-core top-level hierarchy

3.2 Module-by-Module Strategy

For each reference module, the thesis adopts one of four strategies. Table 3.1 summarizes these decisions.

Table 3.1: Thesis strategy per reference module

Module	Strategy	Key Changes
<code>i3c_phy.sv</code>	Adapt	Replace <code>prim_flop_2sync</code> with inline 2FF
<code>bus_monitor.sv</code>	Reuse	Proven edge detection; no changes
<code>bus_tx_flow.sv</code>	Reuse	Bit/byte serialization is well-structured
<code>bus_rx_flow.sv</code>	Adapt	Replace ‘ <code>I3C_ASSERT</code> ’ with standard SVA
<code>bus_tx.sv</code>	Reuse	TX bit engine; direct copy
<code>edge_detector.sv</code>	Reuse	Direct copy
<code>stable_high_detector</code>	Reuse	Direct copy
<code>entdaa_fsm.sv</code>	Improve	Rewrite from target-side to master-side
<code>entdaa_controller.sv</code>	Simplify	Restrict to ENTDAAs only (was 23-state, 40+ CCCs)
<code>flow_active.sv</code>	Improve	Complete all 8 TODO states; add OD/PP; remove HDR
<code>controller_active.sv</code>	Simplify	Remove I2C FSM interface; implement OD/PP mux
<code>I3CCSR.sv</code>	Improve	Replace 14 342-line auto-gen with 266-line manual file
<code>queues.sv</code> (HCI)	Simplify	Replace threshold system with 4 simple sync FIFOs
<code>i3c_wrapper.sv</code>	Improve	Replace AXI4+AHB with simple 32-bit register bus

3.3 Critical Problems in the Reference Design

Several issues in the reference design are directly relevant to the thesis scope.

Unimplemented FSM states. The reference `flow_active.sv` (580 lines) defines 13 FSM states but only 5 are implemented. The remaining 8 states contain only TODO placeholders, making the reference non-functional for any regular I3C or I2C transfer.

Hardcoded open-drain. `controller_active.sv` hardcodes `sel_od_pp_o = 1'b0` (always open-drain), meaning the PHY never switches to push-pull mode. This makes 12.5 MHz SDR operation physically impossible.

Auto-generated register file. `I3CCSR.sv` is 7 710 lines of machine-generated code

including Caliptra-specific registers for secure firmware recovery, standby controller management, and AXI ID filtering — entirely irrelevant to a standalone I3C master controller. The resulting code is unreadable and cannot be meaningfully reviewed.

IBI queue and RX path disabled. `ibi_queue_wvalid_o` and `rx_queue_wvalid_o` are both hardcoded to `1'b0` in the reference, meaning no data is ever written to the receive path.

Platform-specific infrastructure. AXI4 and AHB-Lite adapters with Caliptra privilege enforcement, external SRAM primitives, and 50+ top-level parameters with `#ifdef` conditional compilation — all Caliptra-specific.

3.4 Code Reduction Summary

Table 3.2 compares the code footprint of the reference against the thesis implementation.

Table 3.2: Code reduction: reference vs. thesis

Area	Reference	Thesis	Reduction
Register file	14 342 lines (auto-gen)	266 lines (manual)	98%
Bus interface	~700 lines (AXI+AHB)	~50 lines	93%
HCI queues	2 500+ lines	226 lines	91%
CCC processor	1 406 lines, 23 states	241 lines, ENTDAAs	83%
Command FSM	580 lines, 5 impl.	1 166 lines, 13 impl.	—
Total RTL	~25 000	~4 100	84%

Chapter 4. System Architecture

4.1 Top-Level Block Diagram

The controller is organized as a three-layer hierarchy:

```
i3c_controller_top
+-- i3c_phy          (2FF sync + OD/PP drivers)
+-- csr_register     (32-bit register file + 16-entry
    DAT)
+-- hci_queues       (CMD / TX / RX / RESP FIFOs)
+-- controller_active (protocol engine wrapper)
    +-- flow_active   (13-state command FSM)
        +-- entdaa_controller
            | +-- entdaa_fsm
        +-- bus_tx_flow
        +-- bus_tx
        +-- bus_rx_flow
        +-- bus_monitor
        +-- scl_generator
```

Listing 4.1: Top-level module hierarchy

`i3c_controller_top` is purely structural — no combinational logic, only instantiation and wiring. `flow_active` contains the majority of the original design work.

Figure 4.1 shows the top-level data flow.

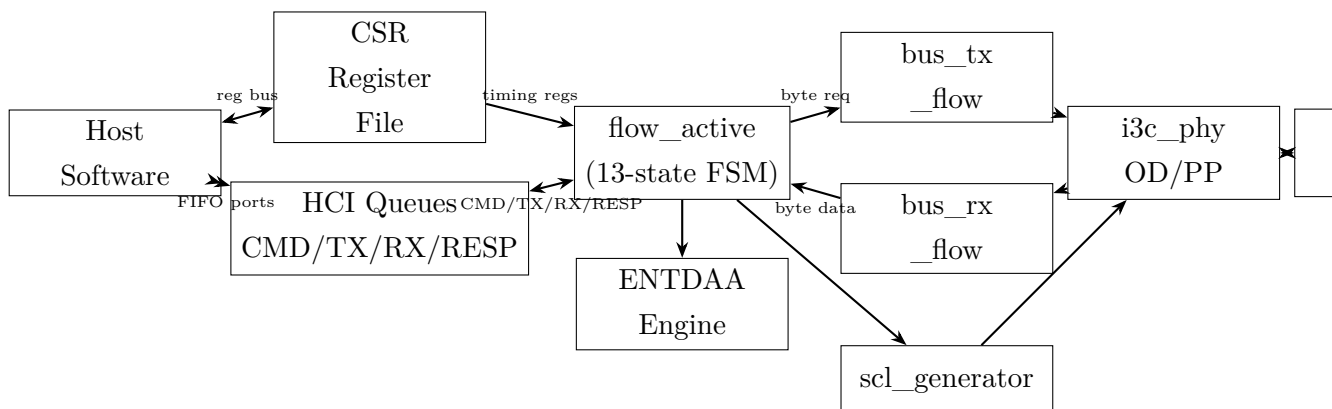


Figure 4.1: Top-level data flow diagram

4.2 Register Interface

The host interface is a simple 32-bit synchronous register bus — no AXI or AHB protocol overhead.

Table 4.1: Register bus interface signals

Signal	Dir.	Width	Description
reg_addr_i	Input	12	Register address
reg_wdata_i	Input	32	Write data
reg_wen_i	Input	1	Write enable
reg_ren_i	Input	1	Read enable
reg_rdata_o	Output	32	Read data
reg_ready_o	Output	1	Transaction complete

4.3 Register Map

Table 4.2: CSR register map

Address	Name	Access	Description
0x000	HC_CONTROL	RW	[0]=ENABLE; [1]=SW_RESET (self-clearing)
0x004	HC_STATUS	RO	[0]=FSM_IDLE; [1]=CMD_FULL; [2]=RESP_EMPTY
0x010–0x030	Timing (×9)	RW	20-bit SCL timing registers
0x100	CMD_QUEUE_PORT	WO	64-bit CMD via two 32-bit writes
0x104	TX_DATA_PORT	WO	Push to TX FIFO
0x108	RX_DATA_PORT	RO	Pop from RX FIFO
0x10C	RESP_PORT	RO	Pop from RESP FIFO
0x110	QUEUE_STATUS	RO	8-bit FIFO flags
0x200–0x23C	DAT[0..15]	RW	32-bit Device Address Table entries

The 64-bit command staging uses two consecutive 32-bit writes: the first write latches `DWORD0`; the second write pushes `{DWORD1, DWORD0}` as a 64-bit command descriptor

into the CMD FIFO.

4.4 HCI Queue Architecture

Four synchronous FIFOs form the data path between the host and the controller:

Table 4.3: HCI FIFO configuration

Queue	Direction	Width	Depth	Purpose
CMD FIFO	SW → HW	64-bit	64	Command descriptors
TX FIFO	SW → HW	32-bit	64	Write data payload
RX FIFO	HW → SW	32-bit	64	Read data payload
RESP FIFO	HW → SW	32-bit	64	Completion status

4.5 Command and Response Descriptors

4.5.1 Command Descriptor (64-bit)

Table 4.4: CMD FIFO descriptor field encoding

Field	Bits	Description
cmd_attr	[2:0]	Type: 000=Regular, 001=Immediate, 010=AddrAssign
tid	[6:3]	Transaction ID (echoed in response)
dat_index	[18:16]	Index into Device Address Table (0–15)
rnw	[28]	0=Write, 1=Read
toc	[30]	Terminate on completion (issue STOP)
cp	[31]	CCC present (prepend 0x7E broadcast header)
data_length	[47:32]	Byte count for regular transfers
imm_data	[63:48]	Inline data for immediate transfers (up to 4 B)

4.5.2 Response Descriptor (32-bit)

Table 4.5: RESP FIFO descriptor field encoding

Field	Bits	Description
<code>data_length</code>	[15:0]	Bytes transferred
<code>tid</code>	[27:24]	Transaction ID (from command)
<code>err_status</code>	[31:28]	Error code

Error codes: 0x0=Success, 0x2=Parity Error, 0x3=Frame Error, 0x4=Address Header Error, 0x5=NACK, 0x6=Overflow.

4.6 Signal Conventions

- Port suffixes: `_i` (input), `_o` (output)
- Reset: active-low asynchronous `rst_ni`
- FIFO handshake: `*_valid` / `*_ready`
- Bus inputs: `scl_i`, `sda_i` (2FF-synchronized)
- Bus outputs: `scl_o`, `sda_o` (active-low open-drain logic)
- `sel_od_pp_o`: 1 = push-pull, 0 = open-drain

Chapter 5. RTL Implementation

5.1 Overview

The RTL is implemented in 18 SystemVerilog source files totaling approximately 4 100 lines, organized across four implementation phases. Table 5.1 lists all files with their line counts.

Table 5.1: RTL source file inventory

File	Lines	Phase
i3c_pkg.sv	172	Phase 0: Packages
ctrl/controller_pkg.sv	20	Phase 0: Packages
phy/i3c_phy.sv	52	Phase 1: Leaf
ctrl/edge_detector.sv	39	Phase 1: Leaf
ctrl/stable_high_detector.sv	44	Phase 1: Leaf
ctrl/bus_monitor.sv	227	Phase 1: Leaf
ctrl/bus_tx.sv	185	Phase 1: Leaf
ctrl/bus_tx_flow.sv	193	Phase 1: Leaf
ctrl/bus_rx_flow.sv	166	Phase 1: Leaf
scl_generator.sv	257	Phase 1: Leaf
hci/sync_fifo.sv	84	Phase 2: Infrastructure
hci/hci_queues.sv	142	Phase 2: Infrastructure
csr/csr_register.sv	266	Phase 2: Infrastructure
ctrl/entdaa_fsm.sv	234	Phase 3: Protocol
ctrl/entdaa_controller.sv	241	Phase 3: Protocol
ctrl/flow_active.sv	1 166	Phase 3: Protocol
ctrl/controller_active.sv	323	Phase 4: Integration
i3c_controller_top.sv	261	Phase 4: Integration
Total	4 372	

5.2 Shared Type Packages

5.2.1 i3c__pkg.sv

Defines top-level shared types including:

- `bus_state_t`: composite struct carrying SCL/SDA value and stability flags from `bus_monitor`.
- `i3c_cmd_attr_e`: 3-bit enum for command type (Regular, Immediate, AddrAssign).
- `i3c_resp_err_status_e`: 4-bit error code enum.
- Command descriptor structs for each command type.
- `localparam logic [6:0] I3C_RSVD_ADDR = 7'h7E`.

5.2.2 controller_pkg.sv

Defines controller-internal types, including the simplified 32-bit `dat_entry_t`:

```

1 typedef struct packed {
2     logic        device;           // [31] 1 = I2C legacy device
3     logic [8:0]   reserved;        // [30:22]
4     logic [6:0]   dynamic_addr;    // [22:16] I3C dynamic address
5     logic [9:0]   reserved2;       // [15:7]
6     logic [6:0]   static_addr;     // [6:0] I2C static address
7 } dat_entry_t;

```

Listing 5.1: Simplified 32-bit DAT entry type

This reduces DAT memory from 128 B (reference 64-bit entries) to 64 B for 16 devices while retaining all required fields.

5.3 Physical Layer

`i3c_phy.sv` (52 lines) handles the physical bus interface with two responsibilities.

Input synchronization: A two-flip-flop synchronizer chains $d \rightarrow \text{ff1} \rightarrow \text{ff2}$ on both SCL and SDA inputs to mitigate metastability at the asynchronous bus/clock-domain boundary.

Output driver: Implements open-drain and push-pull modes under `sel_od_pp_i` control:

- **Open-drain** (`sel_od_pp_i = 0`): SDA driven LOW when `sda_o = 0`, else tri-stated (high-impedance).
- **Push-pull** (`sel_od_pp_i = 1`): active drive of both HIGH and LOW, enabling 12.5 MHz operation.

5.4 SCL Generator

`scl_generator.sv` (257 lines) is a new module implementing a 13-state FSM to generate START, clock, repeated START, and STOP conditions:

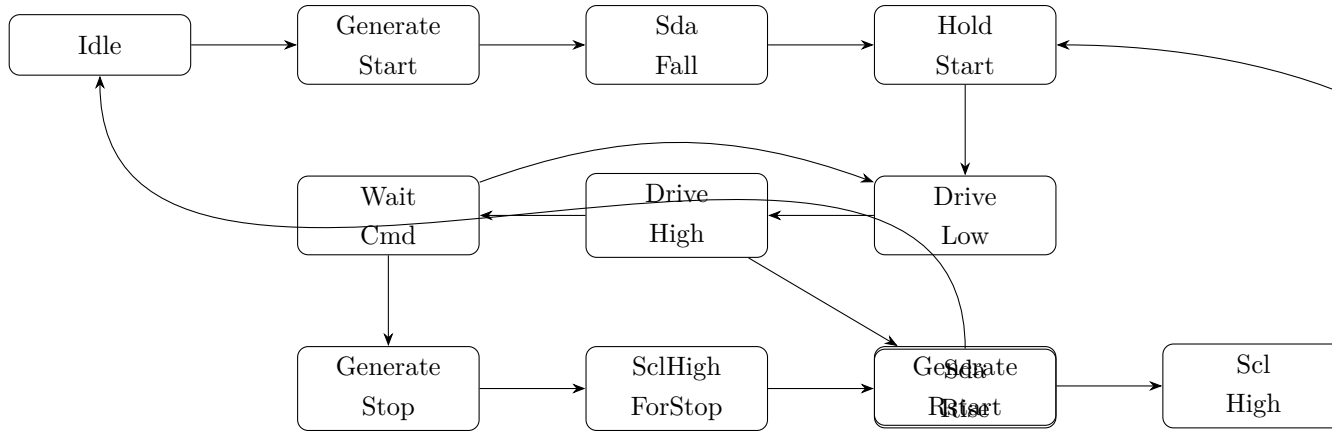


Figure 5.1: SCL generator 13-state FSM

A single countdown counter `tcount` loads a state-dependent initial value from the timing registers and decrements each clock cycle. State transitions occur only when `tcount` reaches zero, precisely meeting I3C/I2C setup and hold time requirements. The selector `sel_i3c_i2c_i` chooses between the I3C (12.5 MHz) and I2C (400 kHz) timing register banks.

5.5 Bus TX and RX Paths

5.5.1 TX Path

`bus_tx.sv` (185 lines) is the bit-level TX engine. It drives SDA synchronized to SCL edges in three steps: (1) wait for `scl_stable_low_i`; (2) set up SDA with the bit value (setup time before SCL rise); (3) hold SDA through SCL HIGH; (4) pulse `done_o` on SCL fall.

`bus_tx_flow.sv` (193 lines) is the byte serializer. A 4-state FSM unpacks a byte, drives bits MSB-first, and generates the T-bit (parity or ACK). The module accepts either full-byte requests (`req_byte_i`) or single-bit requests (`req_bit_i`).

5.5.2 RX Path

`bus_rx_flow.sv` (166 lines) deserializes bits from SDA into bytes. A 4-state FSM samples SDA on each SCL positive edge and assembles 8-bit bytes MSB-first. The received byte and T-bit are available on `rx_data_o` and `rx_t_bit_o`.

5.6 HCI Queues

`sync_fifo.sv` (84 lines) is a parameterized synchronous FIFO using the extra-MSB pointer technique for empty/full detection:

```

1 assign empty_o = (wptr_q == rptr_q);
2 assign full_o  = (rptr_q == {~wptr_q[PtrW], wptr_q[PtrW
    -1:0]});

```

Listing 5.2: FIFO full/empty detection

This technique requires `Depth` to be a power of two, enforced by a compile-time `$fatal` assertion. `hci_queues.sv` (142 lines) instantiates four `sync_fifo` instances with widths and depths from Table 4.3.

5.7 CSR Register File

`csr_register.sv` (266 lines) is a hand-written register file replacing the 14 342-line auto-generated reference. Key implementation points:

- **CMD staging:** `cmd_staging_valid_q` tracks whether `DWORD0` has been latched. On second write, pushes `{DWORD1, DWORD0}` as a 64-bit command descriptor.
- **RX/RESP reads:** assert `rready` to pop the FIFO on the read access cycle, providing zero-latency FIFO access.
- **SW_RESET:** self-clearing; also clears `cmd_staging_valid_q` and `cmd_dword0_q` to prevent stale command corruption.
- **Timing defaults:** preloaded for 333 MHz/I3C 12.5 MHz operation.

5.8 ENTDAEngine

5.8.1 entdaa_fsm.sv

`entdaa_fsm.sv` (234 lines) is a complete rewrite of the reference `ccc_entdaa.sv`. The reference is a *target-side* implementation (transmits 64-bit identity, receives assigned address). The thesis rewrites this as a *master-side* FSM (receives the arbitration bits, transmits the assigned address):

Table 5.2: Master-side ENTDAAs FSM states

State	Action
IDLE	Wait for <code>start_daa_i</code>
WAITSTART	Wait for <code>rstart_det_i</code> (Sr from <code>flow_active</code>)
RECEIVEIDBIT	Assert <code>rx_req_bit</code> ; receive 64 bits (PID + BCR + DCR)
SENDADDR	Assert <code>tx_req_byte</code> ; send {7-bit <code>addr</code> , <code>parity</code> }
WAITACK	Assert <code>rx_req_bit</code> ; read ACK from winning target
DONE	Pulse <code>done_daa_o</code> ; present PID/BCR/DCR outputs
ERROR	On NACK or timeout

5.8.2 entdaa_controller.sv

`entdaa_controller.sv` (241 lines) manages the multi-device ENTDAAs loop. It instantiates `entdaa_fsm` and iterates until all devices in `dev_count_i` have been addressed. For each round, it looks up the pre-assigned address from the DAT and passes it to the FSM.

5.9 Command FSM (`flow_active.sv`)

`flow_active.sv` (1 166 lines) is the most critical module, implementing the 13-state command FSM that orchestrates all master transactions. The reference contained 8 unimplemented TODO states; the thesis implements all 13:

```

1  typedef enum logic [3:0] {
2      Idle,                // Waiting for command
3      WaitForCmd,         // Pop and decode CMD FIFO
4      FetchDAT,           // Look up DAT entry
5      I3CWriteImmediate,  // I3C immediate write (inline data)
6      I2CWriteImmediate,  // I2C immediate write
7      FetchTxData,        // Pop TX FIFO (one DWORD)
8      FetchRxData,        // Push accumulated bytes to RX FIFO
9      InitI2CWrite,       // I2C: START + addr + W + ACK
10     InitI2CRead,        // I2C: START + addr + R + ACK
11     StallWrite,         // Clock-stall: TX FIFO empty
12     StallRead,          // Clock-stall: RX FIFO full
13     IssueCmd,           // Main data transfer workhorse
14     WriteResp           // Push response to RESP FIFO
15 } flow_fsm_state_e;
```

Listing 5.3: flow_active FSM state enumeration

Table 5.3 describes the responsibility of each state. All 8 states that were TODO stubs in the reference are implemented here for the first time.

Table 5.3: flow_active FSM state responsibilities

State	Ref. Status	Implementation
IDLE	Done	Assert <code>fsm_idle_o</code> ; wait CMD
WAITFORCMD	Done	Pop CMD FIFO; decode descriptor
FETCHDAT	Done	Index DAT; wait one cycle for registered read
I2CWRITEIMMEDIATE	Partial	START(OD) + addr + inline bytes
I3CWRITEIMMEDIATE	TODO	START + bcast header + dyn addr + inline bytes with T-bit
FETCHTXDATA	TODO	Pop TX FIFO; go to IssueCmd or StallWrite
FETCHRXDATA	TODO	Write RX DWORD to RX FIFO; continue or stall
INITI2CWRITE	TODO	START(OD) + {static_addr, W} + read ACK
INITI2CREAD	TODO	START(OD) + {static_addr, R} + read ACK
STALLWRITE	TODO	Deassert <code>gen_clock</code> ; poll TX FIFO non-empty
STALLREAD	TODO	Deassert <code>gen_clock</code> ; poll RX FIFO space
ISSUECMD	TODO	Serialize writes; receive reads; ENTDAAs delegation
WRITERESP	Done	Push error code + byte count to RESP FIFO

OD/PP switching is controlled by `sel_od_pp_q`:

- Open-drain: address phase and ACK sampling
- Push-pull: I3C data byte transmission/reception

- Always open-drain: I2C transactions
- Transitions are gated behind `bus_tx_idle` — never mid-byte

Issue phases within `ISSUECMD` are tracked by `issue_phase_q`, an 8-bit counter that sequences through `START` → address → data bytes → `STOP`, keeping TX and RX operations in distinct cycles.

5.10 Controller Wrapper (`controller_active.sv`)

`controller_active.sv` (323 lines) instantiates and connects all protocol sub-modules. Key combinational logic blocks:

SDA multiplexer:

```

1 assign sda_o = scl_gen_busy ? scl_gen_sda_o
2           : tx_active      ? bus_tx_sda_o
3                           : 1'b1; // release bus

```

Listing 5.4: SDA output multiplexer

The SCL generator takes SDA ownership during `START/STOP` generation; `bus_tx` drives SDA during data phases; default is HIGH (bus released).

Arbitration detection:

```

1 assign arbitration_lost = tx_sda_driven & ~bus_state.sda.
   value;

```

Listing 5.5: Arbitration lost detection

Detects when the master drives HIGH but reads LOW on SDA.

Chapter 6. Functional Verification

6.1 Verification Strategy

The verification environment targets functional coverage of all implemented features using the **UVM (Universal Verification Methodology)** framework with **Xcelium** (Cadence, UVM 1.2 CDNS-1.2 bundled) as the simulator.

Verification is structured in two phases:

- **Phase 1 (implemented):** Register agent infrastructure and basic I3C write/read transfers.
- **Phase 2 (planned):** ENTDA sequence, CCC commands, I2C legacy, error injection, multi-device, functional coverage groups.

6.2 Environment Architecture

Figure 6.1 shows the UVM environment hierarchy.

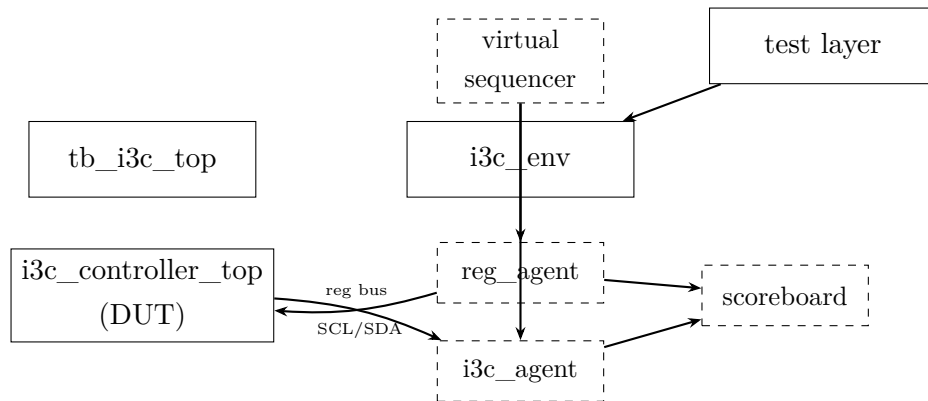


Figure 6.1: UVM environment architecture

6.3 Register Agent

The register agent provides test infrastructure for the simple register bus interface. Table 6.1 lists the Phase 1 components that have been implemented.

Table 6.1: Register agent components (Phase 1, implemented)

File	Description
<code>reg_if.sv</code>	SystemVerilog interface: addr/wdata/wen/ren/rdata/ready
<code>reg_seq_item.sv</code>	UVM sequence item: one register transaction
<code>reg_driver.sv</code>	Drives interface; asserts valid for one cycle per transaction
<code>reg_monitor.sv</code>	Samples completed transactions; sends to analysis port
<code>reg_sequencer.sv</code>	Standard <code>uvm_sequencer</code>
<code>reg_agent.sv</code>	Top-level agent: instantiates driver, monitor, sequencer
<code>reg_agent_cfg.sv</code>	Agent configuration object
<code>reg_agent_pkg.sv</code>	Collects all agent components into one package
<code>i3c_csr_addr_pkg.sv</code>	Named address constants (<code>ADDR_HC_CONTROL</code> , etc.)

6.4 I3C Bus Agent

The I3C agent operates in **device (target) mode** — it responds to master-driven bus activity rather than initiating transfers:

- **Driver:** samples SCL edges; drives SDA during ACK phases and ENTDAAs arbitration.
- **Monitor:** captures complete I3C frames (START to STOP); constructs `i3c_item` objects for the scoreboard.
- **Device response sequence:** automatic ACK/NACK, ENTDAAs PID broadcasting, T-bit generation.

6.5 Scoreboard

The scoreboard performs end-to-end checking:

- **Write check:** compares data observed on the I3C bus against the TX FIFO contents written by the test sequence.
- **Read check:** compares data written to the RX FIFO against data driven by the I3C device driver.

- **Response check:** verifies RESP FIFO `err_status == Success` and `data_length` matches the command descriptor.

6.6 Phase 1 Test Plan

Table 6.2: Phase 1 test scenarios

Test	Sequence	Scenario	Pass Criteria
i3c_smoke	i3c_smoke_vseq	1-byte immediate write	Bus active; RESP=Success
i3c_write	i3c_write_vseq	N-byte regular write via TX FIFO	Bus data = TX data; RESP=Success
i3c_read	i3c_read_vseq	N-byte regular read to RX FIFO	RX data = device data; RESP=Success

The standard test sequence flow is:

1. Configure timing registers (use CSR defaults for 333 MHz / 12.5 MHz)
2. Write DAT entry with device dynamic address
3. Enable controller (`HC_CONTROL[0] = 1`)
4. Write CMD descriptor (two 32-bit writes to `CMD_QUEUE_PORT`)
5. Write TX data to `TX_DATA_PORT` (for write commands)
6. Fork `i3c_device_response_seq` on the I3C bus agent
7. DUT drives: `START` → address → data → `STOP`
8. Poll `HC_STATUS[2]` (`RESP_EMPTY`); read `RESP_PORT`
9. Scoreboard checks all observations

6.7 Build Infrastructure

```

# Compile + elaborate
xrun -compile -elaborate \
    -f verification/uvm_i3c/filelist.f \
    -uvmhome CDNS-1.2

# Run smoke test
xrun -R +UVM_TESTNAME=i3c_base_test \
    +UVM_TEST_SEQ=i3c_smoke_vseq

# Run write test with verbosity
xrun -R +UVM_TESTNAME=i3c_base_test \
    +UVM_TEST_SEQ=i3c_write_vseq \
    +UVM_VERBOSITY=UVM_HIGH

```

Listing 6.1: Xcelium build and run commands

6.8 Phase 2 Roadmap

Table 6.3: Phase 2 verification roadmap

#	Feature	Description
1	ENTDAA test	Full DAA: bcast header → CCC → Sr → arbitration → addr assign
2	Error injection	NACK from target; FIFO overflow; abort conditions
3	CCC tests	ENEC/DISEC broadcast and direct
4	I2C legacy	I2C device write/read at 400 kHz
5	Multi-device	Multiple I3C targets; test DAT lookup
6	Func. coverage	Covergroups: FSM transitions, CCC types, OD/PP, FIFO boundaries

Chapter 7. Bug Analysis and Fixes

7.1 Overview

A systematic code review of all 18 RTL source files was conducted on April 27, 2026, identifying 22 issues classified into four severity levels.

Table 7.1: Bug severity summary

Severity	Count	Criteria
CRITICAL	3	Blocks synthesis or causes total functional failure
HIGH	8	Incorrect protocol or logic behavior
MEDIUM	7	Timing inconsistencies or incomplete features
LOW	4	Style / maintainability
Total	22	

7.2 Critical Issues

BUG-001 — Stray character in CSR parameter list

File: `csr_register.sv`, line 15.

A stray / character after the comma in the parameter list causes a parse error and prevents all compilation:

```
1 // Buggy:
2 parameter int unsigned CmdDataWidth = 64,/
3
4 // Fixed:
5 parameter int unsigned CmdDataWidth = 64,
```

BUG-002 — Missing SCL clock for immediate transfers

File: `flow_active.sv`, states `I2CWRITEIMMEDIATE` and `I3CWRITEIMMEDIATE`.

Impact: all immediate transfers hang after START.

`gen_clock_q` is never asserted in the immediate states. After the START condition, `scl_generator` transitions to `WAITCMD` with `gen_clock=0`, and SCL never toggles.

The TX engine enters TRANSMITDATA and waits indefinitely for SCL edges that never arrive.

Fix: Assert `gen_clock_q=1'b1` at state entry:

```

1 I2CWriteImmediate: begin
2     gen_clock_q  = 1'b1;  // <-- add this
3     sel_i3c_i2c_q = 1'b0;
4     sel_od_pp_q   = 1'b0;
5     // ...
6 end

```

BUG-003 — RX partial DWORD data loss on read transfers

File: `flow_active.sv`, states ISSUECMD and FETCHRXDATA. **Impact:** silent data loss on any read whose length is not a multiple of 4 bytes.

Received bytes are accumulated into `rx_dword_q` four at a time and flushed only on DWORD alignment. When a read completes (`remaining_len_q==0`), the FSM transitions to WRITERESP without flushing any partial DWORD.

Fix: Flush before entering WRITERESP:

```

1 if (remaining_len_q == '0) begin
2     if (rx_byte_idx_q != 2'h0) begin // partial DWORD pending
3         rx_queue_wvalid_q = 1'b1;
4         rx_queue_wdata_q  = rx_dword_q;
5     end
6     state_d = WriteResp;
7 end

```

7.3 High-Severity Issues

BUG-004/005/006 — Same-cycle TX/RX dependency (three instances)

Files: `flow_active.sv`, I2C write (line 997–1011), I2C read (line 1056–1074), I3C read (line 1089–1114).

In all three paths, `bus_rx_req_bit_q` is asserted inside a `bus_tx_done_i` condition block, then `bus_rx_done_i` is checked within the same combinational block. Both signals are single-cycle pulses generated by different modules; they cannot both be asserted in the same cycle.

Impact: ACK/NACK is never sampled for I2C writes, I2C reads, or I3C read T-bit acknowledgement.

Fix: Use `issue_phase_q` to separate TX and RX into distinct cycles:

```

1  // Phase 0: send byte
2  if (issue_phase_q[0] == 1'b0) begin
3      bus_tx_req_byte_q = 1'b1;
4      bus_tx_req_value_q = current_tx_byte;
5      if (bus_tx_done_i) issue_phase_d = issue_phase_q + 1;
6  // Phase 1: receive ACK
7  end else begin
8      bus_rx_req_bit_q = 1'b1;
9      if (bus_rx_done_i) begin
10         nack_detected_d = bus_rx_data_i[0];
11         issue_phase_d = 8'h0;
12     end
13 end

```

BUG-007 — Missing START for I3C regular transfers

File: `flow_active.sv`, `ISSUECMD` state. **Impact:** all I3C regular write/read transfers produce illegal bus activity.

For `RegularTransfer` commands to I3C devices, `ISSUECMD` immediately serializes data bytes without first generating START or transmitting the address byte.

Fix: Add initial `issue_phase` steps in `ISSUECMD` for I3C regular transfers to generate START and send the address byte before data.

BUG-008 — SW reset doesn't clear CMD staging

File: `csr_register.sv`, lines 162–179.

Software reset clears `cmd_wvalid_q` but not `cmd_staging_valid_q` or `cmd_dword0_q`. A stale `DWORD0` after reset will be paired with the next write, producing a garbage 64-bit command.

Fix:

```

1  end else if (sw_reset_q || (cmd_wvalid_q && cmd_wready_i))
2      begin
3          cmd_wvalid_q          <= '0;
4          cmd_staging_valid_q <= '0;  // add
5          cmd_dword0_q         <= '0;  // add
6      end

```

BUG-009 — gen_idle_o never asserted

gen_idle_q defaults to 1'b0 and is never set to 1'b1 in any state, eliminating the only bus-abort mechanism. Without this signal, a stuck SCL generator (e.g., from target clock-stretching) cannot be recovered.

BUG-010 — bus_rx_flow shift register data race

File: bus_rx_flow.sv, lines 54 and 78.

Both the shift register and the output mux use raw sda_i. Since rx_done fires one cycle after the SCL positive edge, sda_i may have been updated to the next bit by the target.

Fix: Use the registered sample rx_bit in both locations.

BUG-011 — FIFO non-power-of-2 depth

The extra-MSB pointer technique silently malfunctions for non-power-of-2 depths. The current default (64) is safe, but the constraint is implicit.

Fix: Add a compile-time assertion:

```

1  initial
2      assert (Depth == (1 << PtrW))
3      else $fatal(1, "sync_fifo: Depth must be a power of 2");

```

7.4 Medium-Severity Issues

BUG-012/013 Off-by-one timing inconsistency between edge_detector (count >= delay) and stable_high_detector (count > delay), producing a one-cycle asymmetry for equal delay_count values.

BUG-014 Missing gen_rstart_i handling in WAITCMD of scl_generator. A repeated START requested while SCL is held LOW will be silently dropped.

BUG-015 ccc_code_o always outputs 0x00 because ccc_code_q is never assigned from the descriptor's cmd field.

BUG-016 T-bit parity check for I3C reads uses a static comparison (bus_rx_data_i[0] != 1'b1) instead of the correct XOR-based odd parity over the received byte.

BUG-017 Bitwise & used instead of logical && in bus_tx.sv line 176.

BUG-018 rx_req_bit registration asymmetry in bus_rx_flow: Idle→ReadBit uses the registered signal, but the abort logic uses the unregistered input.

7.5 Low-Severity Issues

BUG-019 Counter underflow in `entdaa_fsm`: `bit_cnt_d` decrements below zero on the final bit. Benign in current design but fragile if reused.

BUG-020 Signals using `_q` suffix in `flow_active` are assigned combinationaly, violating the project's `_q=registered` naming convention.

BUG-021 Verilog-2001 `wire` keyword used in `controller_active` instead of SystemVerilog `logic`.

BUG-022 ENTDAACCC code hardcoded as `8'h07` in `flow_active` instead of a named constant from `i3c_pkg.sv`.

7.6 Remediation Priority

1. **BUG-001**: Fix first — syntax error blocks all subsequent work.
2. **BUG-002, BUG-003**: Fix before first simulation — makes immediate transfers and non-aligned reads completely non-functional.
3. **BUG-004 through BUG-008**: Fix for correct protocol behavior in all transfer modes.
4. **BUG-009 through BUG-011**: Fix for robustness.
5. **BUG-012 through BUG-022**: Can be deferred but should be tracked.

Chapter 8. Results and Discussion

8.1 Design Metrics

Table 8.1 summarizes the key metrics of the completed design.

Table 8.1: Design metrics summary

Metric	Value
RTL source files	18
Total RTL lines	~4 100
Phase 1 verification files	9
Phase 1 verification lines	~353
Reference design LoC	~25 000
Code reduction vs. reference	~84%
FSM states implemented	13 / 13
FSM states in reference	5 / 13
Bugs identified	22
Critical	3
High	8
Medium	7
Low	4

8.2 Design Contributions

This thesis makes six original contributions relative to the CHIPS Alliance reference.

8.2.1 Complete Command FSM

The thesis is the first complete implementation of the 13-state `flow_active` FSM. The reference left 8 of 13 states as TODO stubs, making it non-functional for regular I3C and I2C transfers, FIFO stalling, and RX data accumulation. The thesis implements all missing states, adding approximately 600 lines of original protocol logic.

8.2.2 Register File Redesign

The 14 342-line auto-generated CSR is replaced by a 266-line hand-written register file — a 98% reduction. The hand-written approach is human-readable, auditable, directly synthesizable, and covers all registers needed for the thesis scope without Caliptra-specific fields.

8.2.3 OD/PP Driver Switching

The reference hardcodes open-drain mode, making 12.5 MHz SDR physically impossible. The thesis implements proper OD/PP switching in `flow_active` (via `sel_od_pp_q`) and routes it through `controller_active` to the PHY — a prerequisite for any I3C SDR data transfer.

8.2.4 Master-Side ENTDAF FSM

The reference `entdaa_fsm.sv` is target-side: it transmits the device’s 64-bit identity and receives an assigned address. The thesis completely rewrites this as a master-side FSM: receiving arbitration identity bits and transmitting the assigned address. All TX/RX roles, counter directions, and state transitions are inverted relative to the reference.

8.2.5 Infrastructure Simplification

AXI4 + AHB adapters (700+ lines) are replaced by a simple register bus (50 lines). The 2 500+ line threshold-based HCI queue system is replaced by four 84-line synchronous FIFOs. Top-level `#ifdef` conditional compilation with 50+ parameters is replaced by a clean parameterized hierarchy.

8.2.6 Systematic Bug Documentation

A formal bug analysis identifies 22 issues with root-cause analysis and recommended fixes, providing a clear remediation roadmap for the verification phase.

8.3 Architectural Trade-offs

Table 8.2: Key architectural decisions and trade-offs

Decision	Choice	Trade-off
Clock domain	Single 333 MHz clock	No CDC complexity; requires fast process
Bus interface	Simple 32-bit register bus	Easy to verify; not AXI-compatible for SoC
DAT depth	16 entries (vs. 128)	Sufficient for thesis; limits bus to 16 devices
I2C timing	Hardcoded 400 kHz	Reduces CSR; no runtime I2C configurability
FIFO type	Synchronous	Eliminates CDC; single-clock design

8.4 Limitations

Simulation not yet run. As of the writing of this report, the RTL has not been simulated end-to-end. The three critical bugs identified prevent correct operation of any transfer. These bugs have been analyzed and fixes documented; applying the fixes and running the Phase 1 UVM tests is the immediate next step.

Verification scope. Only the register agent infrastructure (Phase 1) has been implemented. The full I3C bus agent, scoreboard, and all test sequences remain to be written per the specifications in `docs/verification_specs/`.

FPGA implementation. Synthesis and board-level testing are planned as optional extensions but have not been attempted.

Feature exclusions. IBI, Hot-Join, HDR modes, multi-master, and target mode are architectural exclusions. Adding any requires significant additional modules.

Chapter 9. Conclusion

9.1 Summary

This thesis presents the design of a simplified MIPI I3C Basic v1.1.1 master controller implemented in SystemVerilog, demonstrating the complete frontend IC design pipeline:

1. **Specification study:** thorough analysis of the I3C Basic v1.1.1 standard covering SDR mode, frame format, bus conditions, timing parameters, CCC commands, ENTDAAs protocol, and private read/write transactions.
2. **Reference analysis:** systematic study of the CHIPS Alliance i3c-core identifying reuse, simplification, and improvement strategies for 14 modules. This analysis produced a clear decomposition plan and surfaced critical deficiencies in the reference (8 unimplemented FSM states, hardcoded open-drain, 14 000-line auto-generated CSR).
3. **RTL implementation:** 18 SystemVerilog modules ($\sim 4\,100$ lines) implementing the complete specified feature set. Original contributions include the first complete 13-state command FSM, proper OD/PP switching, a master-side ENTDAAs rewrite, and a 98% smaller hand-written register file.
4. **Verification environment:** UVM-based infrastructure with register agent, CSR address package, and test framework foundation. Verification phase 1 is ready for simulation after bug fixes.
5. **Bug analysis:** 22 issues identified across 18 files, including 3 critical defects. All have root-cause analysis and documented remediation.

9.2 Future Work

1. **Apply bug fixes and run Phase 1 simulations:** Fix BUG-001 through BUG-011 and execute `i3c_smoke`, `i3c_write`, `i3c_read` tests to confirm basic protocol operation.
2. **Complete Phase 2 verification:** Implement the ENTDAAs test sequence, error injection, CCC command tests, multi-device scenarios, and functional coverage groups.

3. **FPGA prototype:** Synthesize to a development board; validate operation with a real I3C target device or protocol analyzer.
4. **Performance analysis:** Measure achieved throughput for I3C SDR (target: 12.5 Mbit/s) and I2C FM (target: 400 kbit/s).
5. **Feature extensions:** IBI and Hot-Join are architecturally natural extensions to add after the base feature set is fully verified.

Bibliography

- [1] MIPI Alliance, *Specification for I3C Basic, Version 1.1.1 with Errata 01*, 2022.
- [2] MIPI Alliance, *Specification for I3C HCI (Host Controller Interface), Version 1.2*, 2022.
- [3] NXP Semiconductors, *UM10204 I2C-bus Specification and User Manual, Rev. 7.0*, 2021.
- [4] CHIPS Alliance, *i3c-core — Open-source I3C Controller/Target Reference Implementation*, <https://github.com/chipsalliance/i3c-core>.
- [5] CHIPS Alliance, *Caliptra — Open-source Root of Trust*, <https://github.com/chipsalliance/caliptra-rtl>.
- [6] Accellera Systems Initiative, *Universal Verification Methodology (UVM) 1.2 User's Guide*, 2015.
- [7] IEEE, *IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2017.
- [8] J. Bergeron *et al.*, *Verification Methodology Manual for SystemVerilog*, Springer, 2005.
- [9] C. Spear and G. Tumbush, *SystemVerilog for Verification: A Guide to Learning the Testbench Language Features*, 3rd ed., Springer, 2012.
- [10] S. Sutherland, S. Davidmann, and P. Flake, *SystemVerilog for Design: A Guide to Using SystemVerilog for Hardware Design and Modeling*, 2nd ed., Springer, 2006.